

Yes. Before thinking about architecture, AI models, databases, or code, the **non-technical problem** is actually quite clear.

The problem in simple terms

Imagine a candidate has **one true resume** containing everything they have genuinely done:

- Education
- Work experience
- Skills
- Projects
- Certifications
- Achievements
- Technologies they've actually used

Let's call this the candidate's "**Source of Truth.**"

Now the candidate finds 20 different jobs.

Each job has a different JD:

Job A wants Python + Django + PostgreSQL
Job B wants Python + FastAPI + AWS
Job C wants Python + ML + TensorFlow
Job D wants Python + Docker + Kubernetes

Today, the candidate has two bad choices.

Option 1 — Manually customize the resume

They create a different resume for every job.

For 20 jobs:

20 JDs → 20 resume modifications → 20 reviews

This takes a lot of time.

Option 2 — Let AI do everything

They give the JD and resume to ChatGPT or another AI tool and say:

"Customize my resume for this job."

This is much faster, but introduces another problem.

AI might say:

"The JD mentions AWS, so let's add AWS to the candidate's skills."

But perhaps the candidate has **never actually used AWS**.

Or AI might turn:

"Worked on a small Python project"

into:

"Designed and implemented a scalable production-grade Python architecture."

That sounds better—but it may not be true.

This creates a very important problem:

Optimization can accidentally become fabrication.

Then comes the ATS problem

There is another layer between the candidate and the recruiter.

The candidate might actually be qualified.

For example:

Candidate genuinely knows:

- Python
- FastAPI
- PostgreSQL
- REST APIs
- Docker

And the JD says:

"Experience developing RESTful web services using Python/FastAPI and PostgreSQL."

The candidate has the required ability.

But suppose their resume says:

"Built backend APIs using Python."

An ATS may have difficulty recognizing the connection because the resume doesn't explicitly contain terms such as:

- FastAPI
- RESTful APIs
- PostgreSQL

So the candidate could be:

Qualified → applies → ATS doesn't recognize enough relevant information → rejected → recruiter never sees them.

That's the real problem you're identifying.

So what should the ideal solution do?

I would describe your idea as:

An AI-powered, candidate-controlled resume optimization system that adapts a candidate's verified profile to each job without inventing information.

The key idea is:

Don't generate a new candidate from every JD.

Instead:

Keep the candidate fixed and adapt the presentation of the candidate.

That's a very important distinction.

Think of it like this

You have:

Candidate



Verified Career Profile



contains everything the candidate can legitimately claim.

Then every JD comes into the system.

JD #1



System identifies what this job wants



Selects relevant information from the candidate's verified profile



Creates **Resume Version #1**

Then:

JD #2



System identifies what this job wants



Selects different relevant information



Creates **Resume Version #2**

But the underlying candidate information **never changes**.

That's the fundamental principle I would recommend.

The "Source of Truth" is the most important part

This should probably be the heart of the product.

Instead of treating the uploaded PDF resume as the only source of information, the system should eventually have a structured candidate profile.

For example:

Skills

Skill	Candidate says	Evidence
Python	Yes	Job A, Project X
FastAPI	Yes	Project X
AWS	Basic	Certification
Kubernetes	No	—
TensorFlow	Yes	Project Y

Now suppose the JD says:

Python, FastAPI, AWS, Kubernetes, TensorFlow

The system can reason:

Python → verified

FastAPI → verified

AWS → verified, but basic

TensorFlow → verified

Kubernetes → not verified

Therefore:

It must NOT simply add Kubernetes.

Instead it should tell the candidate:

Missing requirement: Kubernetes

Your profile does not contain sufficient evidence that you have this skill. We will not add it to your resume.

That is a much safer approach.

This changes the role of AI

The AI shouldn't primarily be:

"Write me a better resume."

Instead, it should be:

"Find the strongest truthful way to represent this candidate for this particular job."

That's a much better product philosophy.

What happens when a JD arrives?

Non-technically, I'd imagine the system doing something like this:

Step 1 — Understand the job

It reads the JD and determines:

- Required skills
- Preferred skills
- Technologies
- Years of experience
- Education requirements

- Responsibilities
 - Certifications
 - Industry/domain experience
 - Important keywords
-

Step 2 — Compare the job with the candidate

It asks:

"What does this candidate actually have that is relevant to this job?"

For example:

JD requires:

Python 
FastAPI 
PostgreSQL 
AWS 
Kubernetes 

Step 3 — Determine what can safely be changed

The system can change things like:

Original

Developed backend applications using Python.

Job-specific version

Developed backend APIs using Python and FastAPI.

Only if the candidate's verified profile supports that claim.

That's legitimate optimization.

But it shouldn't do:

Developed and deployed highly scalable Kubernetes-based microservices.

if the candidate never did that.

The candidate should remain in control

This is another very important part of your idea.

The system shouldn't silently change the resume.

Instead, imagine showing:

Resume Changes

Added

- FastAPI → because it matches a required JD skill
- PostgreSQL → because it is relevant to the position

Reworded

- Project description → to better match the JD terminology

Not added

- Kubernetes → no supporting evidence found

Potential issue

- JD requires 3+ years of experience
- Candidate profile contains 1.5 years

Then the candidate can decide:

Approve

or

Reject

or

Edit

The system could essentially become a "truth firewall"

I think this is one of the strongest aspects of your idea.

You could establish a rule:

AI can rearrange, rephrase, prioritize, and optimize verified information—but cannot create new candidate facts without explicit candidate confirmation.

For example:

Allowed

"Python developer" → "Backend developer specializing in Python"

if supported by the candidate's profile.

Allowed

Moving a relevant project higher on the resume.

Allowed

Using "REST API" instead of a less ATS-friendly equivalent when the candidate genuinely worked with REST APIs.

Not allowed

Adding Kubernetes because the JD mentions Kubernetes.

Not allowed

Increasing 1 year experience to 3 years.

Not allowed

Inventing a project.

Not allowed

Inventing a certification.

Not allowed

Claiming production experience when the candidate only learned something theoretically.

What about the ATS score?

This should also be handled carefully.

I wouldn't position it as:

"Your ATS score is 87%, therefore you'll get the interview."

Because that's misleading.

Different companies use different ATS configurations, ranking mechanisms, screening questions, recruiter workflows, etc.

Instead, your system could say something like:

Job Match Analysis

Overall profile match: 82%

Strong matches

- Python
- FastAPI
- PostgreSQL
- REST APIs

Partial matches

- AWS
- Docker

Missing/unsupported

- Kubernetes

Experience gap

- JD asks for 3+ years
- Candidate has 1.5 years

Resume optimization

- 8 relevant keywords identified
- 6 already represented
- 2 can be truthfully incorporated

That is much more useful than pretending there is one universal ATS score.

And this solves the 20-resume problem

Suppose the candidate has one verified profile.

They apply to:

100 jobs

The system can generate:

100 job-specific resumes

without requiring the candidate to manually rewrite their resume 100 times.

But all 100 resumes originate from the **same verified candidate information**.

That's the key.

The bigger vision

I wouldn't think of this merely as an:

AI Resume Builder

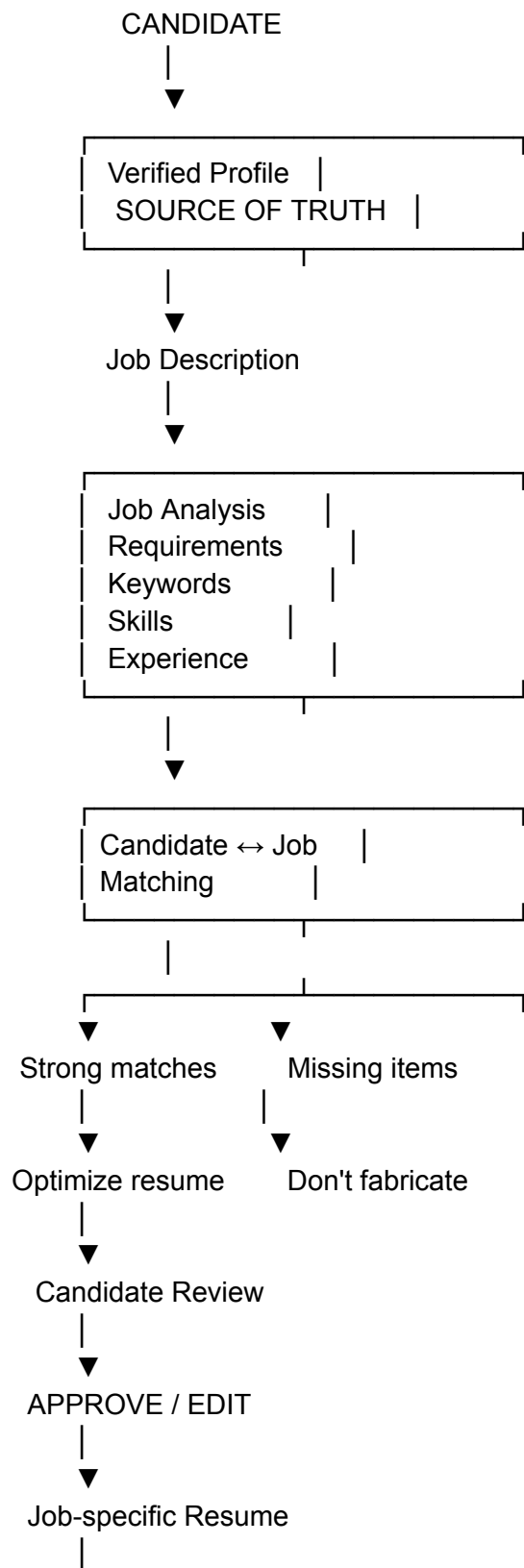
because there are already many products doing that.

Your concept is closer to:

AI Career Profile + Truthful Resume Optimization Engine

The resume becomes an **output** of the candidate's verified profile.

Something like:





There are actually 3 problems you're solving

I would separate your idea into three major problems.

1. Time problem

"I don't have time to customize my resume for every job."

Your system automates customization.

2. Truth problem

"I don't want AI to make things up."

Your system establishes the candidate's verified profile as the source of truth.

3. Visibility problem

"I'm qualified, but the ATS/recruiter isn't recognizing my relevance."

Your system improves the resume's representation of the candidate against the JD.

So the product isn't simply:

"Make my resume prettier."

It's:

"Help me present my real qualifications in the most relevant way for each opportunity."

One important limitation

There is one thing I would **not promise** in the product.

Don't promise:

"We will bypass ATS and guarantee your resume passes ATS."

That's not realistic.

Instead promise something closer to:

"We analyze your resume against the job requirements and improve its relevance, keyword coverage, structure, and readability—without inventing qualifications."

That's both more credible and more defensible.

And auto-apply should come later

I would **not make auto-apply the first/core feature**.

The core product should first prove:

Verified candidate profile → JD analysis → truthful matching → optimized resume → candidate approval

Once that's reliable, auto-apply can become an additional layer:

JD
↓
Analyze
↓
Match candidate
↓
Generate resume
↓
Show changes
↓
Candidate approves
↓
Apply



Record application

Eventually the system could maintain:

Company → Job → JD → Resume version → Changes → Approval → Application status

That would make it much more powerful.

In one sentence

The problem you're identifying can be summarized as:

Candidates need to customize their resumes for every job to maximize ATS relevance, but doing this manually is time-consuming, while fully automated AI customization can introduce inaccurate information.

And the ideal solution is:

A candidate-controlled system that maintains a verified source of truth, analyzes each JD, identifies genuine matches and gaps, generates an ATS-optimized job-specific resume using only supported information, explains every change, and requires candidate approval before submission.

I think **this is a genuinely strong project concept**, especially because it isn't just another "AI resume generator." The interesting part is the **verification + traceability + JD matching + controlled generation**.

If you want to take this forward, the next thing I'd recommend is **not coding yet**. We should first define the product properly: **who the users are, the exact problem statement, user journey, core vs optional features, what "verified information" means, what the system should/shouldn't be allowed to change, and what a realistic MVP looks like.**

Exactly. Before writing a single line of code, we should turn the idea into a well-defined product specification.

I suggest we define it in this order:

1. **Problem Statement — What exact problem are we solving?**
2. **Target Users — Who will use it?**
3. **User Pain Points — What frustrates them today?**
4. **Product Vision — What should the product ultimately become?**

5. Core Value Proposition — Why would someone use it?
6. User Journey — What happens from signup → profile → JD → resume → application?
7. Core Features — What must the product do?
8. MVP Features — What should we build first?
9. Verified Source of Truth — What information can AI use/change?
10. AI Boundaries — What AI is allowed and not allowed to do?
11. ATS Optimization — What exactly are we optimizing?
12. Match/Compatibility Score — How should we calculate and explain it?
13. Candidate Control & Approval — How does the candidate review changes?
14. Auto-Apply Integration — How could it work later?
15. Risks & Failure Cases — Where can the system go wrong?
16. Success Metrics — How do we know the product actually works?
17. MVP vs Future Versions — What should we build now vs later?
18. Business/Real-world Feasibility — Is this actually practical?

Let's start with #1: the Problem Statement

I would define it initially as:

Job seekers increasingly use AI to create and customize resumes for individual job applications. However, customizing a resume for every job is time-consuming, while blindly relying on AI can introduce inaccurate, exaggerated, or unsupported information. At the same time, candidates may be qualified for a position but fail to progress because their resume does not effectively represent the required skills, experience, and keywords for the job description.

The product aims to solve this by maintaining a candidate's verified career information as a single source of truth and intelligently adapting that information to each job description. The system should improve relevance and ATS compatibility without fabricating qualifications, while giving the candidate visibility and control over every meaningful change before the resume is submitted.

The core principle

I'd make this the product's fundamental rule:

AI may optimize how a candidate's experience is presented, but it must not change what the candidate's experience actually is.

This distinction will influence almost everything we build later.

Before we move to architecture

We should now define the product from the user's perspective.

I'll walk you through it step by step, and for each section I'll explain the reasoning rather than just giving you a finished answer.

Next: #2 — Target Users

We'll identify:

- **Primary user**
- **Secondary users**
- **Who has the strongest pain**
- **Who would actually pay/use it**
- **What kind of job seekers we should target for the MVP**
- **Whether freshers, experienced developers, or all candidates should be supported initially**

That decision will significantly affect the rest of the product.

Great. Let's define #2 — Target Users properly before we move forward.

2. Target Users

The biggest mistake would be trying to build this for "everyone looking for a job."

Different candidates have very different problems. For the first version, we should choose a focused audience.

Primary target user

I recommend:

Tech professionals actively applying for jobs, especially software developers, backend developers, data/AI/ML engineers, and other IT professionals with some existing experience.

For example:

- **Python developers**
- **Backend developers**
- **Full-stack developers**
- **Java developers**
- **Data analysts**

- Data scientists
- AI/ML engineers
- DevOps engineers
- QA engineers

Why start with tech?

Because technical JDs contain lots of structured information:

Python, FastAPI, Django, PostgreSQL, AWS, Docker, REST API, Git, React, Kubernetes, etc.

That makes JD → skill matching → resume optimization easier to demonstrate and evaluate.

It also gives us a strong project scope without trying to solve every possible resume problem at once.

Candidate personas

We can think about three important types of users.

Persona 1 — Active job seeker

"I'm applying to 10–30 jobs every week."

Their problem:

- Every JD is different.
- They don't have time to customize resumes.
- They repeatedly edit the same resume.
- They aren't sure whether their resume matches the JD.
- They may not understand ATS optimization.

This should be our #1 target.

Persona 2 — Experienced professional

"I have 3–8 years of experience and a lot of projects/skills."

Their problem is slightly different.

They have too much information.

For example, their master profile might contain:

- **8 technologies**
- **10 projects**
- **4 companies**
- **20 responsibilities**
- **15 achievements**

But a particular JD may only need a subset.

The system should answer:

"Which parts of my experience are most relevant to this particular job?"

This is a very strong use case.

Persona 3 — Fresher / entry-level candidate

Their problem is:

"I don't have enough experience. How do I make my resume relevant?"

This is also valuable, but more complicated.

The system must be especially careful not to turn:

"I learned Docker"

into:

"I have professional Docker experience."

So I would support freshers eventually, but not make them the primary MVP audience.

Who is NOT our initial target?

For the first version, I would avoid trying to optimize for:

- **Executives**

- Academic CVs
- Medical CVs
- Legal professionals
- Highly specialized research positions
- Creative portfolios
- Government applications with unusual formats

These may have completely different requirements.

We can expand later.

The most important user

If we have to choose one person, I'd choose:

A software/IT professional with an existing resume who is actively applying to multiple jobs and wants to customize their resume quickly without allowing AI to invent information.

That's extremely specific—and that's good.

What does this person currently do?

Let's imagine our user.

They have a resume:

Riya — Python Backend Developer

They find this JD:

Python Developer
2+ years experience
Python
FastAPI
PostgreSQL
AWS
Docker
REST APIs

They look at their resume and think:

"I know Python, FastAPI, PostgreSQL and REST APIs, but my resume doesn't highlight them properly."

So they open ChatGPT and give it:

Resume + JD

Then ask:

"Optimize my resume for this job."

The AI creates a new resume.

But now Riya has to check:

- Did AI add something I never did?
- Did it exaggerate my experience?
- Did it remove something important?
- Did it change my project details?
- Did it misunderstand my experience?
- Does the resume actually match the JD?
- Is the formatting ATS-friendly?

And then she has to repeat this process for the next job.

That's the user we're solving for.

What does our product promise this person?

Instead of:

"Give us your resume and we'll magically create a better one."

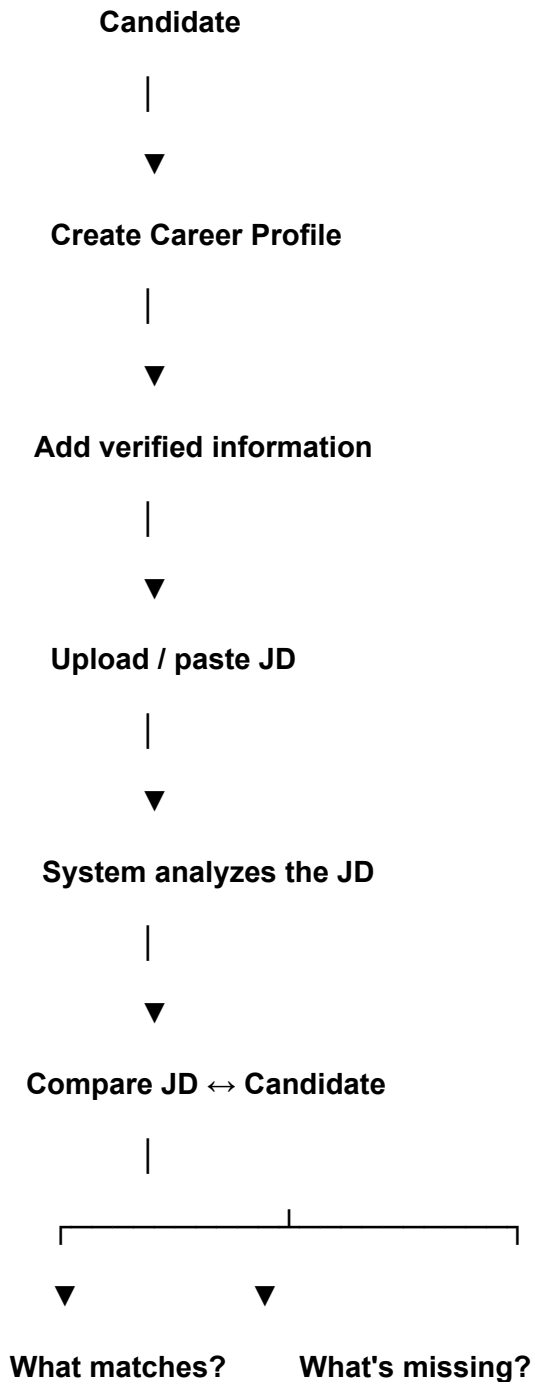
we want:

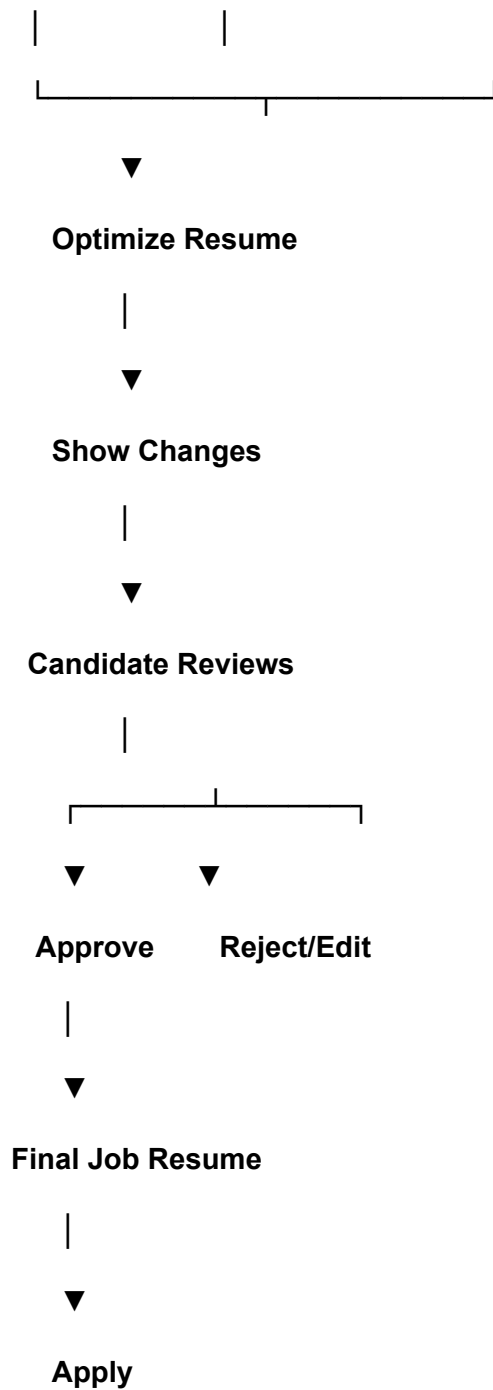
"Tell us what you have actually done once. We'll help you present the right parts of your experience for every job—without making anything up."

That's a much stronger proposition.

Our user journey starts becoming clear

The candidate would eventually do something like:





This is already giving us the skeleton of the product.

One important product decision

There are actually two different things we need to distinguish.

Resume

The document the candidate submits.

Career Profile

The complete, structured collection of everything the candidate can truthfully claim.

For example:

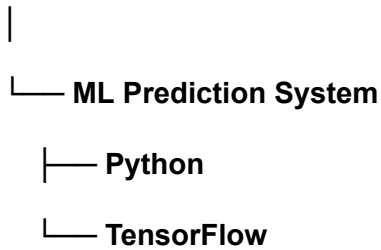
CAREER PROFILE

Experience

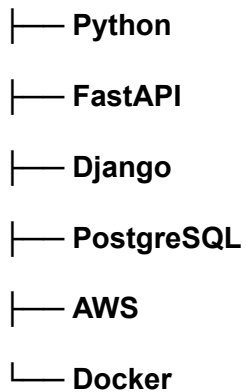
- | — Company A
 - | | — Python
 - | | — FastAPI
 - | | — PostgreSQL
- | — Company B
 - | | — Django
 - | | — AWS

Projects

- | — Document Q&A System
 - | | — Python
 - | | — FastAPI
 - | | — PostgreSQL
 - | | — LLM



Skills



Then:

Career Profile → different resumes

rather than:

Resume → repeatedly modify resume

This is one of the most important design decisions in the whole product.

So our target-user definition

Let's lock this down as our working definition:

Primary User: An actively job-seeking IT professional with an existing resume and verifiable education, skills, projects, and/or work experience who applies to multiple jobs and needs to tailor their resume to different JDs quickly and accurately.

Initial Focus: Software/IT roles, particularly backend, full-stack, Python/Java developers, data, AI/ML, DevOps, and QA roles.

Secondary Users: Freshers and experienced professionals with more complex career histories.

Not initially targeted: Highly specialized CV formats and non-technical professions.

Next: #3 — User Pain Points

Now we should go deeper.

Instead of just saying "customizing resumes takes time," let's identify every problem the candidate experiences before, during, and after applying.

We'll categorize them as:

Before applying → finding a job, understanding the JD

During resume preparation → customization, ATS, AI hallucination

During application → repetitive forms, resume selection

After applying → tracking, knowing why they were rejected

Then we'll identify which problems our product should actually solve and which ones we should deliberately leave out of the MVP.

Absolutely. Let's move to Step 3: User Pain Points.

This step is important because we don't want to build a product based on assumptions. We need to understand exactly what the candidate struggles with, then decide which problems our product will solve.

3. User Pain Points

Let's follow the candidate's journey.

Find Job



Read JD



Understand Requirements



Compare with Own Experience



Customize Resume



Check ATS/Relevance



Review Resume



Apply



Track Application

There are pain points at almost every stage.

3.1 Understanding the Job Description

A candidate sees a JD like:

Python Developer
2+ years experience
Python, FastAPI, PostgreSQL, AWS, Docker
Experience with REST APIs
Knowledge of CI/CD
Strong problem-solving skills

The candidate has to manually figure out:

- **What are the must-have skills?**
- **What are the nice-to-have skills?**

- How many years of experience are required?
- Which technologies are actually important?
- Which keywords are likely important for ATS?
- What responsibilities matter most?
- Which requirements do I actually satisfy?

Pain

The candidate spends time interpreting the JD before even starting the application.

3.2 Understanding Their Own Match

Now the candidate has to ask:

"Do I actually qualify for this job?"

For example:

JD Requirement	Candidate
----------------	-----------

Python	✓ Strong
--------	----------

FastAPI	✓ Strong
---------	----------

PostgreSQL	✓ Strong
------------	----------

AWS	● Basic
-----	---------

Docker	✓
--------	---

Kubernetes	✗
------------	---

3 years
experience

× 1.5
years

The candidate needs to manually make this comparison.

Pain

They don't have a clear picture of their strengths, weaknesses, and gaps relative to the job.

This is something our product can solve very well.

3.3 Resume Customization Is Repetitive

This is the problem you originally identified.

Suppose the candidate applies to:

Job 1 → Backend Developer

Resume emphasizes:

FastAPI + PostgreSQL + APIs

Job 2 → ML Engineer

Resume emphasizes:

Python + ML + TensorFlow + projects

Job 3 → Full Stack Developer

Resume emphasizes:

Python + React + APIs

The underlying candidate hasn't changed.

Only the relevance of different experiences has changed.

But today, candidates often manually create these versions.

Pain

"I have to repeatedly customize essentially the same resume for every job."

3.4 AI Can Introduce False Information

This is one of the most important pain points for our product.

A candidate asks AI:

"Optimize my resume for this JD."

JD says:

Kubernetes experience required.

AI sees Kubernetes and might generate:

"Implemented and managed Kubernetes-based deployments."

But the candidate never did that.

This creates several risks.

Risk 1 — Candidate doesn't notice

They submit the resume without realizing what changed.

Risk 2 — Interview exposes the problem

Recruiter asks:

"Tell me about your Kubernetes deployment architecture."

Candidate:

"Actually... I haven't worked with Kubernetes."

Very bad outcome.

Risk 3 — Trust problem

The candidate can no longer confidently say:

"Everything on my resume is true."

3.5 AI Can Exaggerate True Information

This is slightly different from hallucination.

Suppose the candidate actually did:

"Created a REST API using FastAPI."

AI may turn that into:

"Designed and implemented highly scalable enterprise-grade microservices architecture using FastAPI."

The underlying experience is real, but the claim has been inflated.

So our system needs to distinguish between:

Truthful enhancement

Created REST APIs using FastAPI.

→

Developed RESTful APIs using FastAPI for backend services.

Good.

Exaggeration

Created REST APIs using FastAPI.

→

Architected highly scalable distributed microservices.

Potentially misleading.

3.6 Candidate Doesn't Know What AI Changed

Another major problem.

Imagine AI generates a new resume containing 15 changes.

The candidate may only read the final document.

They don't know:

- What was added?
- What was removed?
- What was rewritten?
- Why was it changed?
- Which JD requirement caused the change?

This creates a black-box problem.

Our product should instead provide something like:

Changed: "Developed backend APIs using Python."
To: "Developed REST APIs using Python and FastAPI."
Reason: FastAPI and REST APIs are required skills in the JD.
Evidence: Project X.

That is much more trustworthy.

3.7 ATS Optimization Is Confusing

Candidates hear things like:

"Your resume needs to be ATS-friendly."

But many don't know what that actually means.

They wonder:

- Which keywords should I include?
- Where should I include them?
- Is my format readable?

- Should I use tables?
- Are my skills being recognized?
- Is my job title relevant?
- Am I missing important terminology?

And there is another problem:

ATS is not one standardized system.

Different employers can use different ATS platforms and configurations.

So our product shouldn't claim:

"Pass every ATS."

Instead:

"Improve your resume's alignment with the requirements and terminology of this job."

3.8 Qualified Candidates Can Still Be Missed

This is the emotional/business pain.

The candidate may think:

"I have the skills. Why am I not getting interviews?"

There could be many reasons, but one possibility is that their resume doesn't clearly communicate their relevance.

For example:

Candidate knows:

PostgreSQL

But resume says:

Worked with relational databases.

Technically related, but less explicit.

Or:

JD:

RESTful API development

Resume:

Developed backend services.

Again, the recruiter/automated system may not immediately recognize the match.

Pain

"My actual abilities aren't being represented effectively."

That's different from lacking skills.

3.9 Applying to Many Jobs Becomes a Full-Time Job

Suppose someone wants to apply to 15 jobs/day.

For each job they might need to:

1. Read JD
2. Analyze requirements
3. Compare their experience
4. Modify resume
5. Check formatting
6. Check keywords
7. Proofread
8. Save a new version
9. Upload resume
10. Fill application
11. Track application

Multiply that by 15.

It's exhausting.

Pain

The job application process itself consumes time that could otherwise be spent preparing for interviews or improving skills.

3.10 Resume Version Management

After applying to many jobs, the candidate can end up with:

resume_final.pdf

resume_final2.pdf

resume_final_new.pdf

resume_python.pdf

resume_python_updated.pdf

resume_backend.pdf

resume_backend_final.pdf

resume_backend_final2.pdf



This sounds small, but it's a genuine workflow problem.

The candidate needs to know:

Which resume did I submit to which company?

And later:

"What exactly did I tell Company X?"

This becomes particularly important when preparing for interviews.

3.11 Candidate Doesn't Know Why They're Weak for a Job

Suppose the system says:

Match: 72%

That's not enough.

The candidate needs:

Why 72%?

For example:

Strong

- Python
- FastAPI
- PostgreSQL
- REST APIs

Partial

- AWS

Missing

- Kubernetes

Experience gap

- Required: 3 years
- Candidate: 1.5 years

Now the candidate has actionable information.

They can decide:

"This job isn't worth applying to."

Or:

"I should apply anyway."

3.12 The Candidate Needs Control

This is perhaps the biggest product philosophy after the Source of Truth.

The candidate shouldn't feel:

"AI is controlling my resume."

They should feel:

"AI is helping me make my resume better."

Therefore:

AI suggests

↓

Candidate reviews

↓

Candidate approves

↓

Resume is generated

This creates trust.

Putting all the pain points together

We can group them into five major categories.

Category

Core Problem

Understanding	Candidate struggles to understand what the JD really requires
Matching	Candidate doesn't know how well their actual experience matches
Customization	Creating a job-specific resume repeatedly takes time
Trust	AI can add/exaggerate unsupported information
ATS visibility	Qualified candidates may not represent their experience using relevant terminology

And there is a secondary workflow problem:

Category	Problem
Version management	Too many resume versions
Application tracking	Difficult to remember what was submitted
Feedback	Candidate doesn't know their gaps
Decision-making	Candidate doesn't know whether a job is worth applying to

Now comes an important product decision

We should NOT try to solve all of these immediately.

If we do, the project becomes:

Resume builder + ATS checker + Job board + Auto-apply + Career coach +
Application tracker + AI interviewer...

That's too much.

Instead, I'd define the core problem as:

"How can we automatically tailor a candidate's resume to a specific job while guaranteeing that the content remains grounded in the candidate's actual experience?"

Everything else should support that.

Core vs Secondary Pain Points

Core — MVP

These directly support our central problem:

1. JD understanding
2. JD ↔ candidate matching
3. Skill/keyword gap identification
4. Truthful resume customization
5. AI hallucination prevention
6. Change explanation
7. Candidate approval
8. Resume generation
9. Match/relevance analysis

Secondary — Later

10. Resume version management
11. Application tracking
12. Auto-apply

- 13. Job discovery
 - 14. Interview preparation
 - 15. Career recommendations
-

Our product is starting to take shape

So far:

Step 1 — Problem

Resume customization is time-consuming, while unrestricted AI customization can be inaccurate and ATS relevance is difficult to manage.

Step 2 — Target user

Actively job-seeking IT professionals applying to multiple jobs.

Step 3 — Pain points

Time + relevance + ATS visibility + AI trust + lack of transparency.

The next step is where this gets interesting:

Let's define Step 4: Product Vision & Value Proposition.

This is where we turn the problems we've identified into a clear answer to:

"What exactly are we building, and why should a candidate use it?"

4. Product Vision

First, let's separate vision from features.

A feature is:

"Generate an ATS-friendly resume."

A vision is:

"Help candidates represent their real professional experience accurately and effectively for every job opportunity."

The second is much bigger and gives us direction for future development.

Our proposed product vision

Build a trusted AI career assistant that helps candidates present their verified skills and experience in the most relevant way for every job—without fabricating, exaggerating, or hiding important information.

There are three important words here:

Trusted

The candidate should be able to trust what the system generates.

Verified

The candidate's actual experience is the foundation.

Relevant

The system adapts that experience to the particular job.

The fundamental idea

Our product should not think like this:

JD

↓

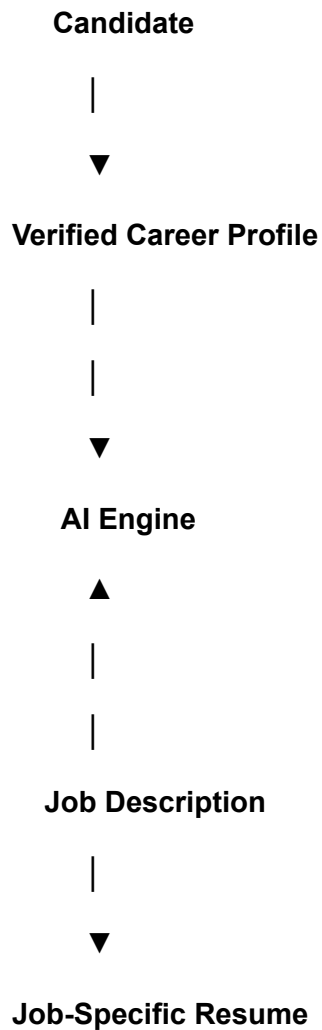
AI

↓

New Resume

Because that's where hallucinations and exaggeration can happen.

Instead:



The candidate and JD are the two inputs.

But the candidate's verified profile controls what AI is allowed to say.

What is our product actually?

I would describe it as:

A job-specific resume optimization platform built around a verified candidate profile.

The resume is not the primary asset.

The candidate profile is the primary asset.

The resume is an output generated from it.

That's a very important product distinction.

Traditional Resume Builder vs Our Product

Consider the difference.

Traditional AI Resume Builder

Candidate Resume

+

Job Description

↓

AI

↓

Optimized Resume

Potential issue:

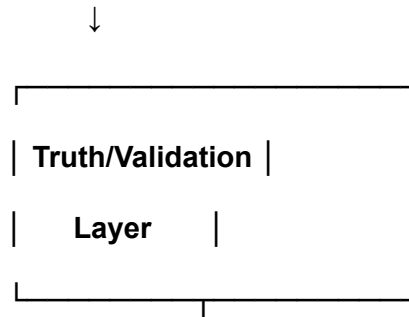
AI may generate information that isn't actually true.

Our approach

Candidate

↓

Verified Career Profile



↓
JD Analysis

↓
Candidate ↔ JD Match

↓
Relevant Information

↓
Resume Optimization

↓
Change Explanation

↓
Candidate Approval

↓
Final Resume

This is the foundation of the product.

The Core Promise

If I were writing the product's promise to users, I'd say:

"Customize your resume for every job without rewriting it from scratch—and without letting AI make things up."

That's simple and directly addresses the pain.

A slightly more professional version:

"Automatically tailor your resume to each job description using only your verified experience, with transparent changes and candidate approval."

I prefer this as the actual product definition.

What makes this different?

There are already tools that do:

- AI resume generation
- Resume rewriting
- ATS checking
- JD matching
- Job application automation

So simply saying:

"We use AI to optimize resumes."

isn't enough.

Our differentiation should be:

Truth + Traceability + Personalization

1. Truth

AI cannot freely invent candidate information.

2. Traceability

Every important change should have a reason.

For example:

Added: FastAPI
Reason: Required by JD
Evidence: Project X

That creates transparency.

3. Personalization

The system doesn't create one generic "perfect resume."

It creates:

The most relevant version of this candidate for this particular job.

Let's define the product in one sentence

Here's the version I'd use for our project documentation:

An AI-powered resume optimization platform that maintains a candidate's verified career profile as a single source of truth, analyzes individual job descriptions, identifies relevant skills and gaps, and generates transparent, ATS-aware, job-specific resumes without introducing unsupported or fabricated information.

That's a strong formal definition.

What does the candidate experience?

Let's imagine you're the candidate.

You open our application for the first time.

Instead of immediately asking:

"Upload your resume."

we eventually want to say:

"Build your Career Profile once. Use it for every application."

You provide your:

Basic information

- **Name**
- **Contact details**
- **Location**
- **Links**

Education

- **Degree**
- **Institution**
- **Graduation year**

Experience

- **Company**
- **Role**
- **Dates**
- **Responsibilities**
- **Achievements**
- **Technologies**

Projects

- **Project**
- **Description**
- **Technologies**
- **Your contribution**

Skills

- **Technical skills**
- **Tools**
- **Frameworks**
- **Languages**

Certifications

- **Certification**

- Issuer
- Date

Achievements

etc.

This becomes your Career Profile.

Then the candidate finds a job

They paste:

Job Description

or eventually:

Job URL

The system analyzes it.

The candidate sees:

Job Analysis

Role: Python Backend Developer

Required

- Python
- FastAPI
- PostgreSQL
- REST APIs
- Docker

Preferred

- AWS
- Kubernetes

Experience

Required: 2+ years

Then the system compares the candidate

For example:

Strong Match

- Python
- FastAPI
- PostgreSQL
- REST APIs

Partial Match

- Docker
- AWS

Missing

- Kubernetes

Experience

Required: 2+ years

Candidate: 1.5 years

Now the candidate knows exactly where they stand.

Then comes the interesting part

The system says:

"I can create a job-specific resume using the following verified information."

It might:

- Move relevant experience higher

- Highlight relevant projects
- Use JD terminology where truthful
- Reorder skills
- Rewrite bullet points
- Remove irrelevant information
- Improve structure
- Improve keyword coverage

But it cannot create new facts.

The candidate sees the changes

For example:

Original

Developed backend APIs using Python.

Proposed

Developed REST APIs using Python and FastAPI.

Why?

The JD specifically requires REST APIs and FastAPI. Your Project X profile confirms experience with both.

Candidate:

✓ Approve

Another change:

AI suggestion

Added Kubernetes experience.

System:

✗ Cannot add

No verified evidence found in your profile.

This is exactly the behavior we want.

Then the candidate gets the final result

Something like:

Job Match

84%

Resume Relevance

91%

Verified JD Keywords

14 / 16

Unsupported Requirements

2

Experience Gap

Yes

But importantly, the system explains these numbers rather than simply throwing a score at the candidate.

The candidate then has a choice

Option A

Download Resume

Option B

Edit

Option C

Approve & Apply

Later, this could connect to an auto-apply workflow.

Where auto-apply fits

Auto-apply shouldn't be the foundation.

It should sit after candidate approval.

Career Profile



Analyze JD



Match



Generate Resume



Explain Changes



Candidate Approval



Submit Application

That gives us an important safety boundary:

No job-specific resume should be submitted without the candidate's approval.

At least for the MVP.

Our product philosophy

I think we should establish these principles now because they'll guide the technical architecture later.

Principle 1 — Candidate owns the truth

The candidate's verified profile is the source of truth.

Principle 2 — AI assists, not invents

AI can transform information but shouldn't fabricate qualifications.

Principle 3 — Every meaningful change is explainable

Candidate should understand what changed and why.

Principle 4 — Candidate remains in control

Nothing important gets submitted silently.

Principle 5 — Relevance over keyword stuffing

We don't simply stuff keywords into a resume.

The goal is to accurately demonstrate relevant experience.

Principle 6 — No guaranteed ATS results

We improve compatibility and relevance; we don't promise that an ATS or recruiter will accept the candidate.

Now let's distinguish our product from "AI Resume Builder"

This is important for your project presentation/interview.

If someone asks:

"Why can't I just use ChatGPT?"

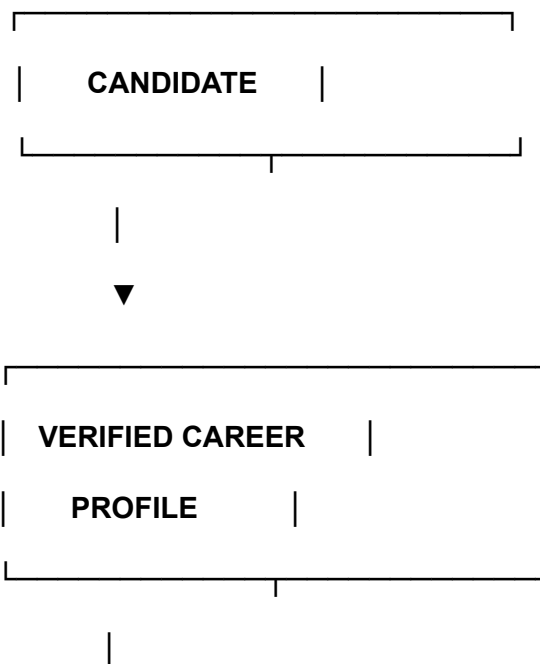
A good answer would be:

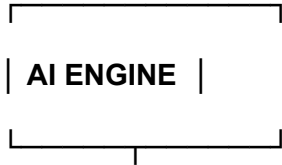
ChatGPT can generate or rewrite a resume when provided with a JD, but it doesn't inherently maintain a structured, verified career profile that acts as a persistent source of truth for the candidate. Our system is designed around that source of truth and adds controlled matching, evidence-based optimization, change tracking, validation, and candidate approval before generating a job-specific resume.

That's a much stronger technical/product justification.

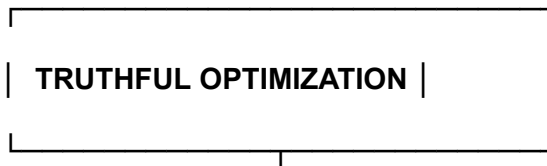
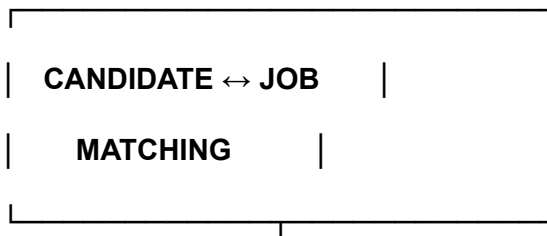
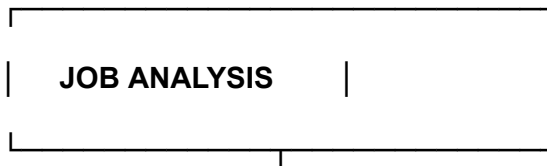
The product's conceptual architecture is now emerging

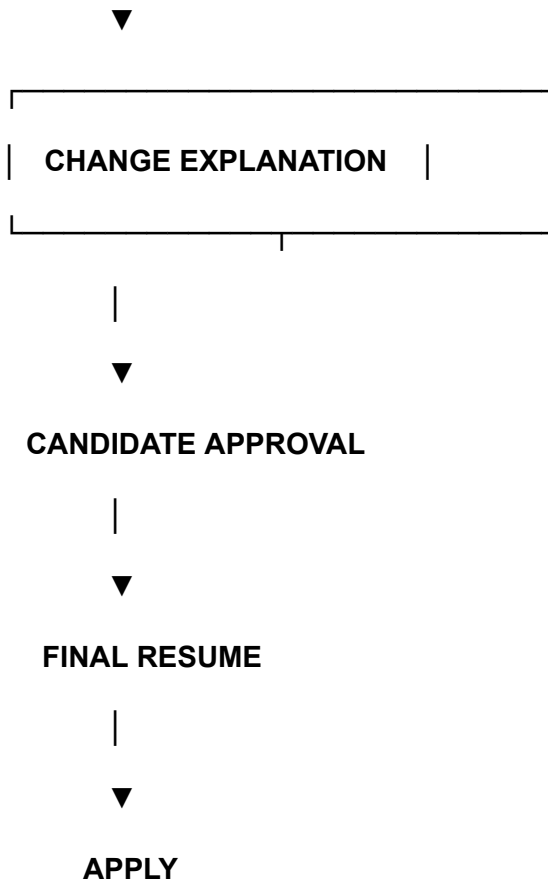
Without getting technical yet:





JOB DESCRIPTION





This is not yet our technical architecture. It's our product model.

That's important—we're defining *what the product does* before deciding *how we'll build it*.

What we've defined so far

Step	Decision
Problem	Resume customization is slow and unrestricted AI can introduce inaccurate information

Primary User **IT professionals actively applying to multiple jobs**

Core Pain **Time + relevance + trust + ATS visibility**

Core Product **Job-specific resume optimization**

Source of Truth **Verified Career Profile**

AI Role **Analyze, match, prioritize, rewrite—not invent**

Transparency **Show what changed and why**

**Candidate
Control** **Approve before finalization/submission**

Auto-Apply **Future/secondary feature**

**Core
Differentiator** **Truth + traceability + personalization**

Next: Step 5 — The Complete User Journey

Now we should design the product as if we were actually using it.

We'll go screen-by-screen / action-by-action:

Signup → Create Career Profile → Verify information → Upload existing resume → Add JD → JD analysis → Match analysis → Optimization → Change review → ATS analysis → Approval → Final resume → Application history

For each step, we'll decide:

- What the candidate sees
- What they provide
- What the system does
- What AI does
- What the candidate controls
- What could go wrong

Once we finish that, we'll have a real product workflow, rather than just an idea.

exactly the right question to ask before building this.

And honestly, if your product is only:

"Give JD + resume → AI suggests changes → ask user for approval → generate resume"

then **you're right**.

A user could simply tell ChatGPT:

"Before making any changes, show me every proposed change and ask for my approval. Never add information that isn't in my resume."

So **that alone is not a strong product**.

The opportunity is to go beyond prompting an LLM.

The key difference: Prompt vs Product

Think about it this way.

ChatGPT approach

You tell the AI:

"Don't hallucinate. Ask for approval."

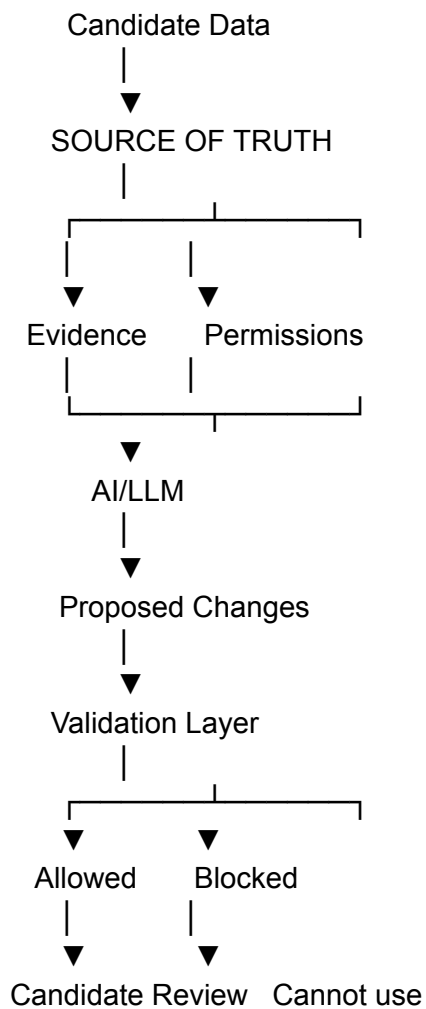
You're **trusting the AI to follow your instruction**.

Your product should instead make this a **system-level rule**.

Your product

The AI isn't the authority.

The system has a separate **candidate source of truth + validation layer**.



The important part is:

The LLM doesn't decide what is true.

Here's a concrete example

Suppose your candidate profile contains:

Skills

Python → Verified
FastAPI → Verified
PostgreSQL → Verified
Docker → Verified
AWS → Basic
Kubernetes → Not verified

And the JD says:

Python, FastAPI, PostgreSQL, AWS, Kubernetes.

A normal ChatGPT conversation might produce:

"You should add Kubernetes to your skills."

You can prompt it not to—but ultimately you're relying on the model.

Your product should say:

Kubernetes

 **Cannot be added**

Reason:

No supporting evidence exists in your verified career profile.

And ideally, there isn't even an "Approve" button for that change.

The candidate would first have to explicitly add/verify the skill.

That's a **product constraint**, not a prompt.

Another important difference: Evidence

This is where I think your idea can become genuinely interesting.

Imagine the candidate has:

Project

AI Document Q&A System

Description:

Built a document Q&A system using Python, FastAPI, PostgreSQL and an LLM.

The system can establish:

FastAPI



Evidence



Project: AI Document Q&A

Now if the JD says:

Experience with FastAPI

the system can tell the candidate:

FastAPI matched

Evidence: AI Document Q&A System

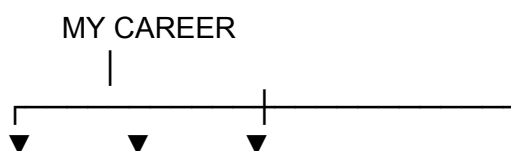
That's considerably stronger than:

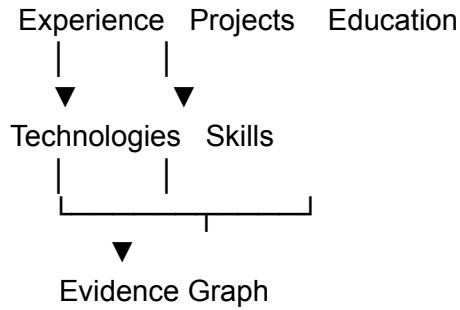
"ChatGPT thinks FastAPI is relevant."

You're basically building a "Career Knowledge Base"

This is the bigger idea.

The candidate enters information **once**.





Then every job is evaluated against that information.

And this solves your original 20-job problem

Imagine:

Candidate

Has 5 years of career information.

Jobs

100 different JDs.

You don't want:

JD 1 → ChatGPT → Resume

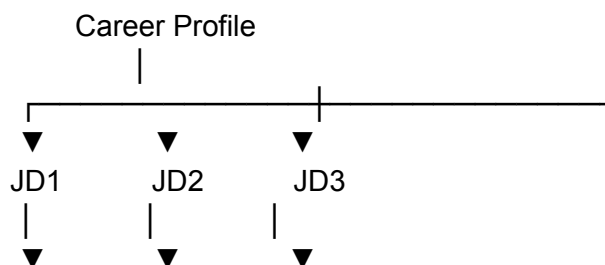
JD 2 → ChatGPT → Resume

JD 3 → ChatGPT → Resume

...

JD 100 → ChatGPT → Resume

You want:



Resume 1 Resume 2 Resume 3

The candidate's career information remains centralized.

There is another major advantage: consistency

Suppose you apply to 50 companies.

Using ChatGPT manually, you might accidentally tell it different things each time.

For example:

Application 1:

Python — 2 years

Application 17:

Python — 3 years

Application 35:

Python — 2.5 years

Now your application history becomes inconsistent.

Your product can enforce:

Python experience = X

unless the candidate explicitly changes their career profile.

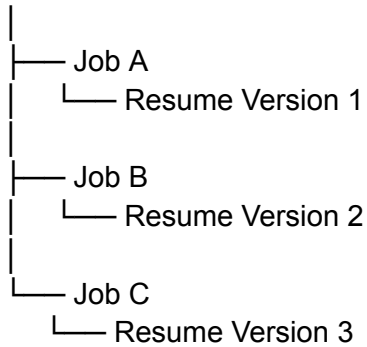
So every generated resume is based on the same underlying truth.

And then there's version/history

This is something ChatGPT isn't fundamentally designed around.

Your product could maintain:

Candidate Profile



Candidate can later ask:

"What resume did I submit to Company B?"

And the system knows.

It could show:

Company B — Backend Developer

Resume:

Version 2

Changes:

- FastAPI moved to prominent position
- Backend project prioritized
- ML project shortened
- PostgreSQL keyword added from verified project evidence

That becomes valuable during interview preparation too.

Here's where your product can become much bigger

Instead of thinking:

AI Resume Generator

think:

AI Career Operating System

The resume is only one output.

Your central asset is:

Verified Career Profile

From that you can eventually generate:

Resume

↓

Cover Letter

↓

Job Match Analysis

↓

Application

↓

Interview Preparation

↓

Interview Questions

↓

Skills Gap

↓

Career Recommendations

All based on the same verified information.

The really interesting part: "Why am I a match?"

Imagine the candidate gets:

You are a strong match for this job.

But the system can actually explain:

JD Requirement



Python



Candidate Evidence



Company X — Backend Developer



Worked with Python for API development

Or:

JD Requirement



FastAPI



Candidate Evidence



Project: AI Document Q&A



Used FastAPI for backend APIs

That's much more useful than a generic AI-generated resume.

And there's a second side to this

Your product shouldn't only ask:

"What keywords can we add?"

It should ask:

"Does the candidate actually satisfy this requirement?"

That's a fundamental difference.

For example:

JD

3+ years Python experience

Candidate

1.5 years professional experience

- 1 year academic/project experience

Your product should say:

Experience mismatch

The position requires 3+ years. Your profile currently supports approximately 1.5 years of professional experience.

And **not** manipulate the resume to make it look like 3 years.

That is valuable.

So let's test our idea honestly

If our product only has:

- Resume generator
- JD analyzer
- "Approve changes" button
- ATS score

 **I would NOT build it.**

ChatGPT and existing resume tools can already do much of that.

But if our product has:

1. Persistent Career Profile

Candidate enters career information once.

2. Evidence-backed information

Every skill/experience can be linked to supporting evidence.

3. Controlled AI

LLM cannot independently introduce unsupported claims.

4. JD ↔ Career matching

System determines what the candidate actually matches.

5. Truthful optimization

AI changes presentation, not facts.

6. Explainable changes

Every meaningful change has a reason/evidence.

7. Gap detection

Shows what the candidate genuinely lacks.

8. Resume versioning

Tracks which resume was used for which job.

9. Application history

Connects JD → resume → application.

10. Candidate-controlled approval

Candidate has final authority.

11. Eventually auto-apply

But only using an approved resume.

Then we have a much stronger product.

One more important point

There's a potential **business moat** here.

A prompt can be copied in 30 seconds.

For example:

"Don't hallucinate and ask me for approval."

Anyone can copy that.

But it's much harder to replicate:

Candidate Profile → Evidence → Skill normalization → JD parsing → Matching → Validation → Resume generation → Change tracking → Version history → Application history

That is a **system**, not a prompt.

And that's what we should build.

So I would modify our original product definition

Instead of:

"AI-powered ATS resume optimizer"

I'd position it as:

A trusted career intelligence platform that maintains a candidate's verified professional profile and uses it to intelligently match,

customize, and optimize job applications without fabricating candidate information.

The first product could focus on resumes, but the underlying architecture should be designed around the **Career Profile**.

And this changes Step 5

Before we design the user journey, I think we should actually define one additional thing:

What exactly counts as "verified information"?

Because this is the foundation of our entire differentiation.

For example, should the candidate simply say:

"I know Kubernetes"

and we mark it as verified?

Or should we distinguish between:

- **Self-declared**
- **Evidence-backed**
- **Professionally verified**
- **AI-inferred**
- **Unverified**

And what is AI allowed to do with each category?

I think we should settle this before moving to Step 5, because this will determine whether our "truth layer" is actually meaningful or just another prompt saying "don't hallucinate."